Introduction to Input/Output in Java

Input/Output (I/O) refers to the process of receiving data from an input source and sending data to an output destination.

Examples of input sources include the keyboard, files, and network connections. Examples of output destinations include the screen, files, and network connections.

Java provides I/O classes mainly through the `java.io` package. Modern Java also provides file-handling facilities through `java.nio.file`.

Types of I/O operations

- Input: Reading data from a source.
- Output: Writing data to a destination.
- File handling: Creating, reading, writing, and managing files.
- Console I/O: Receiving keyboard input and displaying output.

Advantages of file handling

1. Data can be stored permanently.
2. Information can be retrieved later.
3. Large amounts of data can be processed.
4. Data can be shared between program executions.
5. Files can be used to store records, reports, and configuration information.

Concepts of Streams in Java

A stream is a flow of data between a source and a destination. Java uses streams to perform input and output operations.

For example, when a program reads a text file, data flows from the file into the program. When a program writes to a file, data flows from the program to the file.

Types of streams

Java's traditional I/O streams are divided into two main categories:

Java I/O Streams

Byte Streams

Handle raw 8-bit bytes; useful for binary data.

Examples: `InputStream`, `OutputStream`

Character Streams

Handle text through character encoding and decoding.

Examples: `Reader`, `Writer`

Byte streams

Byte streams transfer data in units of bytes. They are commonly used for images, audio, video, and other binary files.

Main abstract classes:

- `InputStream`
- `OutputStream`

Character streams

Character streams are designed for text. They convert between bytes and characters using a character encoding where required.

Main abstract classes:

- `Reader`
- `Writer`

Important exam point: Use byte streams for raw binary data and character streams for text data.

Stream Classes in Java

Java provides several stream classes for reading and writing data.

Main stream class hierarchy

Byte Stream Classes

`InputStream`

Reads bytes

`FileInputStream`, `BufferedInputStream`, `DataInputStream`, `ObjectInputStream`

`OutputStream`

Writes bytes

`FileOutputStream`, `BufferedOutputStream`, `DataOutputStream`, `ObjectOutputStream`

Character Stream Classes

`Reader`

Reads characters

`FileReader`, `BufferedReader`, `InputStreamReader`, `StringReader`

`Writer`

Writes characters

`FileWriter`, `BufferedWriter`, `OutputStreamWriter`, `StringWriter`

Common stream classes

Class	Purpose
`FileInputStream`	Reads bytes from a file
`FileOutputStream`	Writes bytes to a file
`BufferedInputStream`	Buffers byte input for efficiency
`BufferedOutputStream`	Buffers byte output for efficiency
`DataInputStream`	Reads primitive data types from a stream
`DataOutputStream`	Writes primitive data types to a stream
`ObjectInputStream`	Reads serialized objects
`ObjectOutputStream`	Writes serialized objects
`FileReader`	Reads characters from a file
`FileWriter`	Writes characters to a file
`BufferedReader`	Buffers character input and supports `readLine()`
`BufferedWriter`	Buffers character output
`InputStreamReader`	Converts byte input into characters using a charset
`OutputStreamWriter`	Converts characters into bytes using a charset
`StringReader`	Reads characters from a string
`StringWriter`	Collects character output in a string buffer

Byte Stream Classes

Byte streams are used to read and write data as individual bytes. They are suitable for binary files that should not undergo text decoding or encoding.

The two fundamental abstract classes are:

- `InputStream` — the base class for byte input.
- `OutputStream` — the base class for byte output.

FileInputStream

`FileInputStream` reads raw bytes from a file.

Example: Reading a file

Suppose a file named `input.txt` contains:

```
Hello Java
```

Java program:

```
import java.io.FileInputStream;
import java.io.IOException;

class Main {
    public static void main(String[] args) {
        try (FileInputStream in =
                 new FileInputStream("input.txt")) {
```

```
            int data;

            while ((data = in.read()) != -1) {
                System.out.print((char) data);
            }

        } catch (IOException e) {
            System.out.println("File reading failed.");
        }
    }
}
```

Output:

```
Hello Java
```

Explanation:

- `FileInputStream` opens the file for reading.
- `read()` returns one byte at a time as an integer.
- It returns `-1` when the end of the stream is reached.
- The example casts each byte to a character, which is suitable for this simple ASCII text but is not a general method for decoding arbitrary text.

FileOutputStream

`FileOutputStream` writes bytes to a file.

Example: Writing to a file

```
import java.io.FileOutputStream;
import java.io.IOException;
import java.nio.charset.StandardCharsets;

class Main {
    public static void main(String[] args) {
        String text = "Welcome to Java";

        try (FileOutputStream out =
                 new FileOutputStream("output.txt")) {

            out.write(text.getBytes(StandardCharsets.UTF_8));
            System.out.println("Data written successfully.");

        } catch (IOException e) {
            System.out.println("File writing failed.");
        }
    }
}
```

Output:

```
Data written successfully.
```

The file `output.txt` will contain:

```
Welcome to Java
```

Important: By default, `FileOutputStream` overwrites an existing file. To append data, use `new FileOutputStream("output.txt", true)`.

Buffered byte streams

`BufferedInputStream` and `BufferedOutputStream` use an internal buffer to reduce the number of direct I/O operations.

Example:

```
import java.io.*;

class Main {
    public static void main(String[] args) {
        try (BufferedInputStream in =
                  new BufferedInputStream(
                      new FileInputStream("input.bin"))) {

            int data;

            while ((data = in.read()) != -1) {
                System.out.print(data + " ");
            }

        } catch (IOException e) {
            System.out.println("I/O error occurred.");
        }
    }
}
```

This program prints the numeric value of each byte in `input.bin`.

Character Stream Classes

Character streams are designed for reading and writing text. They use the abstract classes `Reader` and `Writer`.

Unlike byte streams, character streams perform character decoding or encoding when connected to byte-based sources and destinations.

FileReader

`FileReader` reads text from a file.

Example

```
import java.io.*;

class Main {
    public static void main(String[] args) {
        try (FileReader reader = new FileReader("input.txt")) {
            int ch;

            while ((ch = reader.read()) != -1) {
                System.out.print((char) ch);
            }

        } catch (IOException e) {
```

```
            System.out.println("Unable to read file.");
        }
    }
}
```

Output (if the file contains `Hello Java`):

```
Hello Java
```

`FileReader` uses a character encoding determined by its constructor or the platform default, depending on the Java version and API used. Where predictable encoding is important, an explicit charset is preferable.

FileWriter

`FileWriter` writes characters to a text file.

Example

```
import java.io.*;

class Main {
    public static void main(String[] args) {
        try (FileWriter writer = new FileWriter("message.txt")) {
            writer.write("Java File Handling");
            System.out.println("Text written successfully.");

        } catch (IOException e) {
            System.out.println("Unable to write file.");
        }
    }
}
```

Output:

```
Text written successfully.
```

The file `message.txt` will contain:

```
Java File Handling
```

By default, this constructor overwrites existing file contents. An append option is also available.

BufferedReader

`BufferedReader` improves character-input efficiency and provides the convenient `readLine()` method.

Example

```
import java.io.*;

class Main {
    public static void main(String[] args) {
        try (BufferedReader br =
                 new BufferedReader(new FileReader("input.txt"))) {
```

```
            String line;

            while ((line = br.readLine()) != null) {
                System.out.println(line);
            }

        } catch (IOException e) {
            System.out.println("Error reading text.");
        }
    }
}
```

This program reads a text file one line at a time until the end of the file is reached.

BufferedWriter

`BufferedWriter` buffers character output and provides methods such as `write()` and `newLine()`.

```
import java.io.*;

class Main {
    public static void main(String[] args) {
        try (BufferedWriter bw =
                 new BufferedWriter(new FileWriter("student.txt"))) {

            bw.write("Name: Amit");
            bw.newLine();
            bw.write("Course: BCA");

            System.out.println("File created successfully.");

        } catch (IOException e) {
            System.out.println("Writing error.");
        }
    }
}
```

Output:

```
File created successfully.
```

Contents of `student.txt`:

```
Name: Amit
Course: BCA
```

String-Based Stream Classes

The topic “byte string classes” may refer to byte stream classes and string stream classes. In Java, the main string-based character stream classes are `StringReader` and `StringWriter`.

StringReader

`StringReader` reads characters from a `String` as though the string were a character stream.

```
import java.io.*;

class Main {
```

```
    public static void main(String[] args) {
        String text = "Hello Java";

        try (StringReader reader = new StringReader(text)) {
            int ch;

            while ((ch = reader.read()) != -1) {
                System.out.print((char) ch);
            }
        } catch (IOException e) {
            System.out.println("Reading error.");
        }
    }
}
```

Output:

```
Hello Java
```

StringWriter

`StringWriter` collects character output in an internal string buffer instead of writing to a file.

```
import java.io.*;

class Main {
    public static void main(String[] args) {
        StringWriter writer = new StringWriter();

        writer.write("Hello ");
        writer.write("Java");

        System.out.println(writer.toString());
    }
}
```

Output:

```
Hello Java
```

Difference between `StringReader` and `StringWriter`

`StringReader`	`StringWriter`
Reads characters from a string	Collects characters in a string buffer
Used as an input stream	Used as an output stream
Extends `Reader`	Extends `Writer`
`read()` retrieves characters	`toString()` returns collected text

Other Important Stream Classes

DataInputStream and DataOutputStream

These classes allow Java programs to read and write primitive data types in a defined binary format.

Example:

```
import java.io.*;

class Main {
    public static void main(String[] args) {
        try (DataOutputStream out =
                  new DataOutputStream(
                      new FileOutputStream("data.bin"))) {

            out.writeInt(100);
            out.writeDouble(25.5);
            out.writeBoolean(true);

        } catch (IOException e) {
            System.out.println("Writing error.");
        }

        try (DataInputStream in =
                  new DataInputStream(
                      new FileInputStream("data.bin"))) {

            System.out.println(in.readInt());
            System.out.println(in.readDouble());
            System.out.println(in.readBoolean());

        } catch (IOException e) {
            System.out.println("Reading error.");
        }
    }
}
```

Output:

```
100
25.5
true
```

The values must be read in the same order and with the same types in which they were written.

ObjectInputStream and ObjectOutputStream

These classes can read and write objects using Java serialization.

For serialization, the object's class generally implements `Serializable`.

Example:

```
import java.io.*;

class Student implements Serializable {
    private static final long serialVersionUID = 1L;
    int rollNo = 101;
    String name = "Amit";
}

class Main {
    public static void main(String[] args) {
        try (ObjectOutputStream out =
                  new ObjectOutputStream(
                      new FileOutputStream("student.dat"))) {

            out.writeObject(new Student());
```

```
        } catch (IOException e) {
            System.out.println("Object writing failed.");
        }
    }
}
```

This program writes a serialized `Student` object to `student.dat`. Object input/output should only be used with appropriate trust and security precautions when reading serialized data.

Using Streams in Java

Streams are used for several common operations, including reading from the keyboard, displaying output, reading files, writing files, and copying data.

Standard input and output streams

Java provides three standard streams through the `System` class.

Stream	Purpose
`System.in`	Standard input, usually the keyboard
`System.out`	Standard output, usually the console
`System.err`	Standard error output, usually the console

Example: Reading keyboard input

```
import java.util.Scanner;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your name: ");
        String name = sc.nextLine();

        System.out.println("Hello, " + name);
        sc.close();
    }
}
```

Sample output:

```
Enter your name: Rahul
Hello, Rahul
```

`Scanner` is a convenient class for parsing input. It is not itself a subclass of `InputStream`; it reads from the stream supplied to its constructor.

Copying data from one file to another

The following program copies the bytes of one file into another file.

```
import java.io.*;

class Main {
    public static void main(String[] args) {
        try (
```

```
            FileInputStream in =
                new FileInputStream("source.bin");
            FileOutputStream out =
                new FileOutputStream("copy.bin")
        ) {
            byte[] buffer = new byte[4096];
            int count;

            while ((count = in.read(buffer)) != -1) {
                out.write(buffer, 0, count);
            }

            System.out.println("File copied successfully.");

        } catch (IOException e) {
            System.out.println("File copying failed.");
        }
    }
}
```

Output if the operation succeeds:

```
File copied successfully.
```

Explanation:

1. The source file is opened for reading.
2. The destination file is opened for writing.
3. A buffer temporarily stores the bytes read from the source.
4. Each group of bytes is written to the destination.
5. The try-with-resources statement closes both streams automatically.

This program copies raw bytes, making it suitable for binary files as well as text files.

The importance of closing streams

Streams should be closed when they are no longer needed to release resources and ensure buffered output is flushed.

Java provides try-with-resources, which closes resources automatically.

```
try (FileReader reader = new FileReader("input.txt")) {
    // Read file contents
} catch (IOException e) {
    System.out.println("I/O error.");
}
```

This approach is usually preferable to manually closing a stream in a `finally` block.

Difference Between Byte Streams and Character Streams

Byte streams	Character streams
Operate on bytes	Operate on characters
Based on `InputStream` and `OutputStream`	Based on `Reader` and `Writer`
Suitable for binary files	Suitable for text files

Do not perform text decoding by themselves	Decode or encode text when connected to byte-based sources or destinations
Examples: `FileInputStream`, `FileOutputStream`	Examples: `FileReader`, `FileWriter`

Advantages of Java I/O Streams

1. Support reading and writing data from various sources.
2. Provide separate classes for binary and text operations.
3. Support buffering for more efficient I/O.
4. Support primitive data and serialized objects.
5. Allow streams to be chained together to add functionality.
6. Support resource management through try-with-resources.

Other Useful I/O Classes in Java

Java provides several classes in the `java.io` package to read, write, buffer, and manage data.

Class	Use
`File`	Represents a file or directory path and provides file-management operations
`BufferedInputStream`	Reads bytes efficiently using a buffer
`BufferedOutputStream`	Writes bytes efficiently using a buffer
`BufferedReader`	Reads text efficiently, including line by line
`BufferedWriter`	Writes text efficiently
`DataInputStream`	Reads primitive data types such as `int`, `double`, and `boolean`
`DataOutputStream`	Writes primitive data types in binary form
`ObjectInputStream`	Reads serialized objects
`ObjectOutputStream`	Writes serialized objects
`PrintStream`	Prints formatted values; `System.out` is an example
`PrintWriter`	Writes text conveniently using methods such as `print()` and `println()`
`InputStreamReader`	Converts bytes into characters using a character encoding
`OutputStreamWriter`	Converts characters into bytes using a character encoding
`StringReader`	Reads characters from a string
`StringWriter`	Writes characters into an in-memory string buffer

Example: Using `PrintWriter`

```
import java.io.*;

public class PrintExample {
    public static void main(String[] args) throws IOException {
        try (PrintWriter pw =
                  new PrintWriter("message.txt")) {
            pw.println("Welcome to Java");
            pw.println("File handling example");
        }

        System.out.println("Data written successfully.");
    }
}
```

Output:

```
Data written successfully.
```

The file `message.txt` contains:

```
Welcome to Java
File handling example
```

Note: `PrintWriter` is convenient for text output. Check `checkError()` if you need to detect write errors because many of its write methods do not throw `IOException` directly.

Using the `File` Class in Java

The `File` class belongs to the `java.io` package. It represents the pathname of a file or directory and provides methods to check, create, delete, and inspect filesystem entries.

Important: Creating a `File` object does not automatically create a physical file.

Syntax

```
import java.io.File;

File f = new File("sample.txt");
```

Important methods of the `File` class

Method	Purpose
`exists()`	Checks whether the path exists
`createNewFile()`	Creates a new empty file if it does not exist
`mkdir()`	Creates one directory
`mkdirs()`	Creates directories, including missing parent directories
`isFile()`	Checks whether the path represents a regular file
`isDirectory()`	Checks whether the path represents a directory
`getName()`	Returns the file or directory name
`getPath()`	Returns the path as supplied
`getAbsolutePath()`	Returns the absolute path
`length()`	Returns file size in bytes
`canRead()`	Checks whether the file can be read
`canWrite()`	Checks whether the file can be written
`delete()`	Attempts to delete a file or an empty directory
`list()`	Returns names inside a directory
`renameTo()`	Attempts to rename or move a filesystem entry

Program: Using the `File` class

```
import java.io.*;

public class FileClassDemo {
    public static void main(String[] args) throws IOException {
        File f = new File("student.txt");

        if (f.createNewFile()) {
            System.out.println("File created.");
        } else {
            System.out.println("File already exists.");
        }
```

```
        System.out.println("Name: " + f.getName());
        System.out.println("Path: " + f.getAbsolutePath());
        System.out.println("Exists: " + f.exists());
        System.out.println("Is file: " + f.isFile());
    }
}
```

Possible output (the file did not previously exist):

```
File created.
Name: student.txt
Path: [absolute path to student.txt]
Exists: true
Is file: true
```

The absolute path depends on the computer and the directory from which the program is run.

Input/Output Exceptions in Java

An I/O exception is an exception that occurs while performing input/output operations, such as reading a file, writing data, opening a file, or accessing a stream.

Most I/O operations use `IOException` or one of its subclasses.

Important I/O exceptions

Exception	Meaning
`IOException`	General input/output failure
`FileNotFoundException`	File cannot be opened or created as requested, commonly because it does not exist or access is denied
`EOFException`	Unexpected end of input while reading structured data
`UTFDataFormatException`	Invalid modified UTF-8 data encountered in certain data-stream operations
`UnsupportedEncodingException`	Requested character encoding is not supported by the relevant API

Other exceptions may also occur. For example, `SecurityException` can arise from access restrictions, and `InvalidClassException` may occur during object deserialization.

Handling I/O exceptions

Java provides three common approaches:

1. `try-catch` — handles an exception locally.
2. `throws` — declares that a method may pass an exception to its caller.
3. `finally` — runs cleanup code in most circumstances, whether or not an exception occurs.

Example using `try-catch`

```
import java.io.*;

public class ExceptionDemo {
```

```
    public static void main(String[] args) {
        try {
            FileReader fr = new FileReader("abc.txt");
            fr.close();

            System.out.println("File opened successfully.");
        } catch (FileNotFoundException e) {
            System.out.println("File not found or cannot be opened.");
        } catch (IOException e) {
            System.out.println("I/O error occurred.");
        }
    }
}
```

Possible output (if the file cannot be opened):

```
File not found or cannot be opened.
```

The `FileReader` constructor can throw `FileNotFoundException`, while `close()` can throw `IOException`.

Best practice: try-with-resources

Try-with-resources automatically closes resources that implement `AutoCloseable`, including most file streams and readers.

```
import java.io.*;

public class SafeFileRead {
    public static void main(String[] args) {
        try (BufferedReader br =
                  new BufferedReader(new FileReader("abc.txt"))) {

            String line;
            while ((line = br.readLine()) != null) {
                System.out.println(line);
            }

        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Advantages:

- Automatically closes the file.
- Reduces resource leaks.
- Handles read and close errors.
- Makes code easier to maintain.

Creation of a File in Java

A file can be created using the `createNewFile()` method of the `File` class.

Syntax

```
File f = new File("data.txt");
f.createNewFile();
```

The method returns `true` if a new file is created and `false` if the file already exists. It can throw `IOException`.

Program

```
import java.io.*;

public class CreateFile {
    public static void main(String[] args) {
        try {
            File f = new File("data.txt");

            if (f.createNewFile()) {
                System.out.println("File created successfully.");
            } else {
                System.out.println("File already exists.");
            }
        } catch (IOException e) {
            System.out.println("Unable to create file.");
        }
    }
}
```

Output on first run:

```
File created successfully.
```

Output on a subsequent run:

```
File already exists.
```

The file is created in the program's working directory unless another path is specified.

File Handling and CRUD Operations

CRUD represents the four basic operations used to manage stored data.

C — Create

Create a new file or create a new data record.

R — Read

Read and display the contents of a file.

U — Update

Modify existing data or append new content to a file.

D — Delete

Remove a file or delete a particular record from stored data.

For basic Java file-handling questions, CRUD can be demonstrated with `File`, `FileWriter`, `FileReader`, and `BufferedReader`.

1. Create operation

Create a new file using `createNewFile()`.

```
File f = new File("record.txt");

if (f.createNewFile()) {
    System.out.println("Created");
}
```

2. Read operation

Read the file line by line using `BufferedReader`.

```
import java.io.*;

public class ReadFile {
    public static void main(String[] args) {
        try (BufferedReader br =
                 new BufferedReader(new FileReader("record.txt"))) {

            String line;
            while ((line = br.readLine()) != null) {
                System.out.println(line);
            }

        } catch (IOException e) {
            System.out.println("Read error: " + e.getMessage());
        }
    }
}
```

If `record.txt` contains `Hello Java`, the output is:

```
Hello Java
```

3. Update operation

There are two common forms of updating a text file.

Append data: Use `FileWriter` with `true` to add content at the end without overwriting existing content.

```
import java.io.*;

public class AppendFile {
    public static void main(String[] args) {
        try (FileWriter fw = new FileWriter("record.txt", true)) {
            fw.write("\nNew record added");
            System.out.println("Data appended.");
        } catch (IOException e) {
            System.out.println("Write error: " + e.getMessage());
        }
    }
}
```

Overwrite existing content: Use `new FileWriter("record.txt")` without `true`. This replaces the existing contents when the file is opened for writing.

```
try (FileWriter fw = new FileWriter("record.txt")) {
```

```
    fw.write("Updated record");
}
```

For editing a particular line or record, a simple approach is to read the existing contents, modify them in memory, and write the revised contents back. For large files or concurrent updates, a more robust storage method is preferable.

4. Delete operation

Use the `delete()` method of the `File` class.

```
import java.io.*;

public class DeleteFile {
    public static void main(String[] args) {
        File f = new File("record.txt");

        if (f.delete()) {
            System.out.println("File deleted.");
        } else {
            System.out.println("Unable to delete file.");
        }
    }
}
```

Possible output:

```
File deleted.
```

`delete()` returns a boolean. A `false` result means the deletion did not succeed; check whether the path exists, whether permissions allow deletion, and whether the target is a non-empty directory.

Complete CRUD Program in Java

The following program demonstrates all four CRUD operations on a text file. It creates the file if needed, reads its contents, appends data, and deletes it.

```
import java.io.*;

public class FileCRUD {
    public static void main(String[] args) {
        File file = new File("student.txt");

        // CREATE
        try {
            if (file.createNewFile()) {
                System.out.println("1. File created.");
            } else {
                System.out.println("1. File already exists.");
            }
        } catch (IOException e) {
            System.out.println("Create error: " + e.getMessage());
            return;
        }

        // CREATE / WRITE CONTENT
        try (FileWriter fw = new FileWriter(file)) {
            fw.write("Name: Rahul\n");
```

```
            fw.write("Course: BCA\n");
            System.out.println("2. Initial data written.");
        } catch (IOException e) {
            System.out.println("Write error: " + e.getMessage());
            return;
        }

        // READ
        System.out.println("3. Reading file:");
        try (BufferedReader br =
                 new BufferedReader(new FileReader(file))) {
            String line;
            while ((line = br.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            System.out.println("Read error: " + e.getMessage());
            return;
        }

        // UPDATE: APPEND CONTENT
        try (FileWriter fw = new FileWriter(file, true)) {
            fw.write("Year: First Year\n");
            System.out.println("4. Data updated.");
        } catch (IOException e) {
            System.out.println("Update error: " + e.getMessage());
            return;
        }

        // READ UPDATED CONTENT
        System.out.println("5. Updated contents:");
        try (BufferedReader br =
                 new BufferedReader(new FileReader(file))) {
            String line;
            while ((line = br.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            System.out.println("Read error: " + e.getMessage());
            return;
        }

        // DELETE
        if (file.delete()) {
            System.out.println("6. File deleted.");
        } else {
            System.out.println("6. File could not be deleted.");
        }
    }
}
```

Expected output on a fresh run

```
1. File created.
2. Initial data written.
3. Reading file:
Name: Rahul
Course: BCA
4. Data updated.
5. Updated contents:
Name: Rahul
Course: BCA
Year: First Year
6. File deleted.
```

If `student.txt` already exists, the first message will instead say that the file already exists. The program overwrites its initial content before appending the updated record.

Important Differences

`File` class	File stream classes
Represents a file or directory path	Read or write actual data
Checks existence and file properties	Transfers bytes or characters
Provides `createNewFile()` and `delete()`	Provides methods such as `read()` and `write()`
Does not itself read file contents	Performs data input/output

`FileReader`	`FileInputStream`
Character-oriented input	Byte-oriented input
Suitable for text files	Suitable for binary files and raw bytes
Reads characters	Reads bytes

`FileWriter`	`FileOutputStream`
Character-oriented output	Byte-oriented output
Suitable for text	Suitable for binary data
Supports text writing	Writes raw bytes